Scan order of the n = 83 K600 one-target index calculus

2026-10-15

Scan order of the n = 83 K600 one-target index calculus

Status (2026-10-15): source audit and receipt reconstruction done; protocol frozen and Tier 1 run started; no timing result yet. The Tier 1 run is GitHub Actions run 38786433493, started by the push of commit `ba49f0854` to branch `claude/n83-scan-order` (workflow `.github/workflows/n83-scan-order.yml`). Every number below is one of four things: an exact count, a reconstruction of an archived receipt, a labelled model estimate, or an earlier experiment's figure. None is a timing measurement of this experiment. The verdict, the new baseline `S` and the gaps to tuned rho and to BL are **pending**. They will come from `verify_receipt.py` on the archived receipt, under the rule in `protocol.json`.

Question

The finding comes from stage 0 of the quotient law (`claude/ic-probe-latency`, `experiments/ic-quotient-law-stage0-20261015/`, commit `cfa806a38`). On B's 16 fresh points, the K600 cached-log baseline used 2.30× [0.66, 4.27] the probes predicted by `r/(A·#states)`. Its CV was 1.57 and the KS test gave p = 0.001. Three points took 14.5–18.1M probes, against a law mean of 1.73M. Every other base follows the geometric law.

That raises two questions:

- Is the scan order the cause?
- Can an exact change to the order lower the current baseline's one-target online time?

This is an iteration under AGENTS.md "Success criterion: iterative improvement of the index-calculus baseline" (PR #919).

What the source and the receipts show (exact)

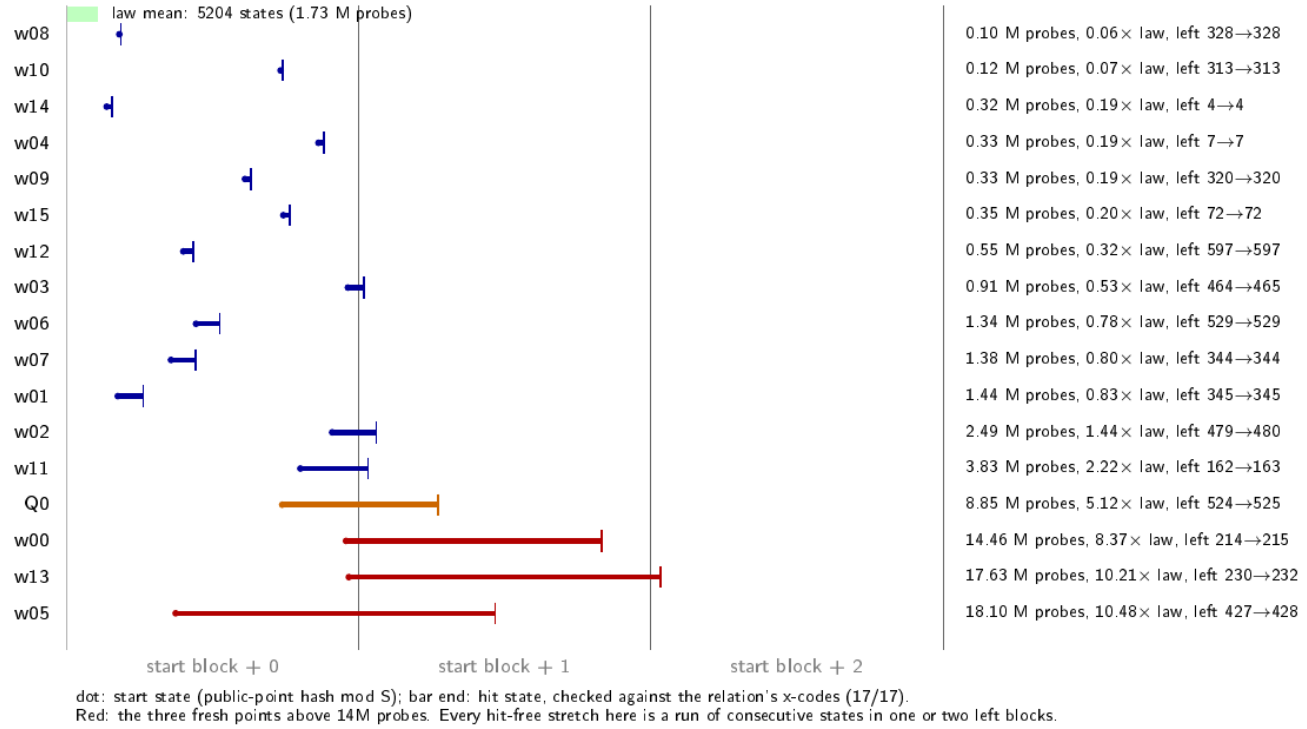


Figure 1: Every archived baseline scan, start to hit, in left-orbit blocks. Data: `analysis/receipt_blocks.json` from `receipt_blocks.py` over BL-arm run 38689817618 (source, vector).

The source is crypto `770f6a69` `examples/koblitz_orbit_dlp_fast_online.rs`, `mod wide`, plus B's `ic_log_cache.patch`:

- `build_index128` pushes one state for every (left, right, relative), in that loop order. Every pair gives a state, because its two roots are $x(A \pm \tau^{rel} B)$. The receipts confirm this: `regular_states` is 29,880,000 = 600·600·83. The state vector is therefore **left-orbit-major**, with 49,800 states per left orbit.
- `extract128_timed` scans `states[start..]` and then `states[..start]`. Here `start` is the public-point hash mod S.
- `scan_state128` batch-solves 2n = 166 roots per state and probes two partners per root, so one state is 332 probes. A left block is 16,533,600 probes, against a law mean of 1,727,752.

The receipts. `receipt_blocks.py` recomputes each point's start state from the point alone, and its states scanned from `root_calls / 166`. It takes the first IC receipt per point of BL-arm run 38689817618, copied into `inputs/bl_run_38689817618_ic/`. It then checks the implied hit state against the relation's own x-codes: left orbit, right orbit and relative shift. All 17 of 17 match. So every archived baseline scan is reconstructed exactly, state by state.

point	probes	× law	start block, states left in it	hit block, offset	hit-free states before the hit	geometric-law tail prob.
w00	14,455,639	8.37	214, 2,219	215, 41,322	43,541	2.3e-4
w05	18,098,258	10.48	427, 31,284	428, 23,228	54,512	2.8e-5
w13	17,634,153	10.21	230, 1,745	**232** (231 empty), 1,569	53,114	3.7e-5
Q0	8,845,441	5.12	524, 13,103	525, 13,539	26,642	6.0e-3
other 13 fresh	0.10–3.83M	0.06–2.22	—	10 of 13 in the start block	—	≥ 0.11

What this establishes:

- **The mechanism is confirmed: the tail is a scan-order artefact.** A geometric probe count cannot produce hit-free runs of 43–55 thousand consecutive states, about 10 law means, in 3 of 16 points. The left-major scan produces them by walking through sparse left blocks.
  - Each four-term representation Q = ±A ± B ± C ± D is found at the 12 ordered states over its four orbits. That puts 3 hits in each of 4 left blocks.
  - So the hit density varies by left block, and a scan stays in one block for 16.5M probes.
  - w13 crossed the whole of block 231 without a hit.
- **The arithmetic of the hypothesis is refuted.** "About 20% of targets find no hit in the first block and pay a whole extra block" is not what happened:
  - Six of 16 fresh points hit past their start block. Five of them started within 10,005 states of that block's end (1,745 to 10,005 states left).
  - Only one point (w13) paid a whole empty block.
  - The receipts show runs through *sparse* blocks, not one-block quanta.
- **A model, labelled as such.**
  - Assume each target has Poisson(λ = C(99,600, 4)/r = 478) representations, each one hitting 3 states in each of its 4 blocks.
  - Under that model, a left-major scan from a uniform start costs **1.33–1.36**× the law (CV 1.33). A uniformly random state order costs **0.99**× (CV 0.99).
  - The rank stage measured 1.40× over its 600 relations (run 38523707244), close to the model.
  - The model makes three tails in 16 points unlikely (p ≈ 0.002). Either the 16-point panel is an unlucky draw, or the clustering is stronger than the model. Both readings favour a random order.

The fix

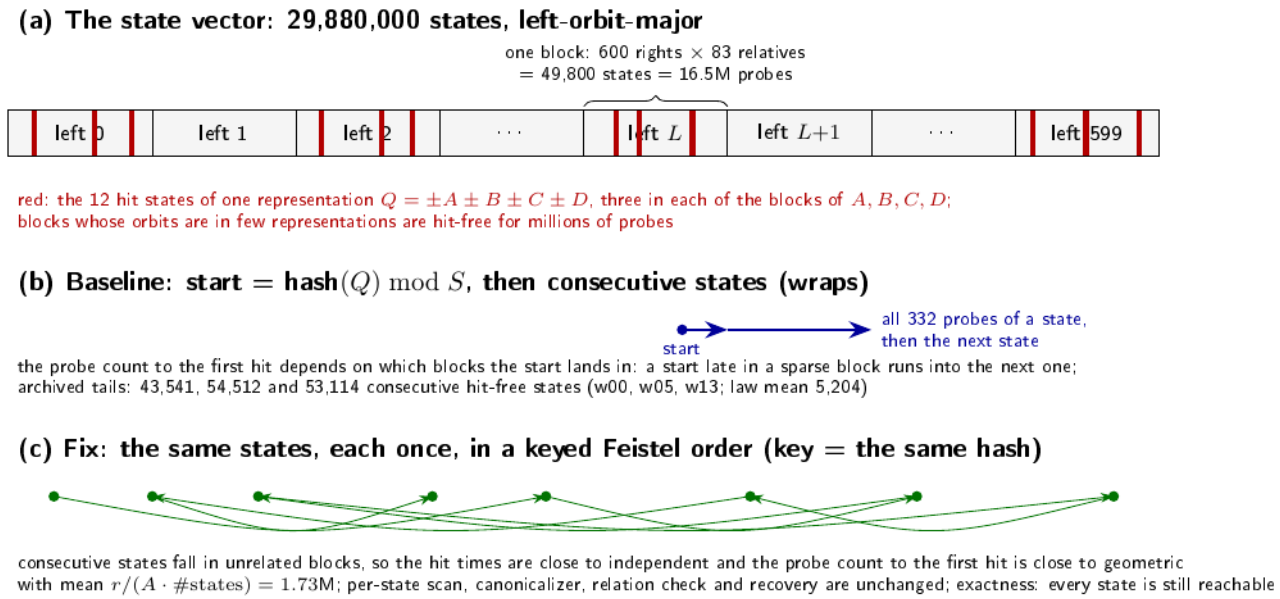


Figure 2: The state vector, the baseline order and the Feistel order (schematic; the hit marks are illustrative, the data are in the figure above) (source, vector).

`make_online_source.py` derives the producer from B's patched producer by counted, exact string replacements in `mod wide`. It adds a compile-time state-order selector, `KIC_SCAN_ORDER`:

- **`left_major`.** This is the default, and the rank stage always uses it. It is B's code path unchanged. The gate requires it to reproduce B's archived probe count and relation on all 17 points.
- **`feistel`.** It visits the same 29,880,000 states, each exactly once, in the order of a keyed four-round balanced Feistel bijection on [0, 2^26). Images ≥ S are skipped.

  - The key is the same public-point hash that B uses as `start`.
  - The order is computed inside the timed interval, one evaluation per index, so no work moves out of the interval.
  - Everything per state is B's: the 166 batched root calls, the full 83-rotation canonicalizer `NormalBasis128::canonical` on every partner, the root-table lookup, `lift128` and the descent.
  - The canonicalizer is deliberately left at the baseline's full scan. `lift_scan` (`claude/n83-liftscan`, run 38776966036) touches the same path and is measured separately.
- **Exactness.**
  - The order is a bijection, so the search covers the same states. It finds a relation exactly when B's search does.
  - Every relation is still the first one that `lift128` verifies. Its scalar comes from the same verified factor-log table and is replayed as $[d]G = Q$.
  - The DLP is unique, so the recovered scalars must be identical across orders. The relation and the probe count differ, which is the point of the change.
  - Setup self-tests the bijection on the real S with two keys, in every process and outside the interval. The crate tests check every length 1..2100 under five keys, and the real S.

Frozen design

protocol.json (with its power check) and schedule.json were committed before any timing. `restore.py` and `make_schedule.py --check` re-verify every digest on the runner, as do `make_heldout_targets.py--check` and the regeneration of `inputs/planted_starts.json`. The run uses one GitHub-hosted `ubuntu-24.04` runner, with every process pinned to one CPU under the `isolated_bench` Tier 1 counters. Rows the time guard skips are written as `skipped_time_guard`.

Arm	Order	Role
`fB` / `qB`	left-major	baseline on B's 16 fresh points / on Q0
`fP` / `qP`	Feistel	candidate
`fBaa` / `qBaa`	left-major	A/A control
`fH` / `qH`	left-major, `KIC_SCAN_START_OVERRIDE`	planted positive control: starts halfway through B's own hit-free stretch, so it must report exactly the predicted probes (`inputs/planted_starts.json`) and B's relation, at about half the work
`rT`	—	A's tuned rho, rT770 (Fold/64/DP6), on Q0 (seed 202610060 + block) and each fresh point (seed 830000 + w)
`hB` / `hP`	left-major / Feistel	secondary held-out panel: 384 new public points (role `scanorder-heldout`), in six 64-point processes per arm, with one online interval per target

- **Order.** Q1, F1, Q2, F2, H1, Q3, F3, H2, . . . , Q7, F7, H6, Q8, F8. Within each segment the order is a seeded shuffle.
- **Primary (the rule as given).** The §23 ratio of means, fB/fP, on online time over the 16 fresh points, with a bootstrap over points (10,000 draws).
  - **Improvement** if the lower bound is above 1.
  - **Regression** if the upper bound is below 1.
  - Otherwise **not_separated**.
  - Both A/A controls (fresh and Q0) must lie inside [0.97, 1.03], and both planted lower bounds must exceed 1. Otherwise the verdict is `invalid`.

  - An improvement makes the Feistel order the new one-target baseline.
- **Also reported.**
  - Geometric means of probes and of online time for each arm, and their ratio, with a bootstrap.
  - Mean probes over the law, and the CV.
  - Q0 per-block ratios.
  - The held-out panel's §23 ratio and geometric-mean ratio.
  - `S_probe` and `S_rho`, the online cost in this job's tuned-rho steps over $\sqrt{r}$, for the old and new baselines, with their deltas.
  - The tuned-rho gap on the same points.
  - The BL gap. It is **derived across experiments**: this job's IC/rT times BL run 38689817618's rT/BL of 232.1 on the same 16 points. It is distance only.
- **Power, checked before freezing** (`power_check.py`, `analysis/power.json`). Probes stand in for time.
  - **The primary has about 11% power.** The 16-point baseline mean is carried by three tail points, so a bootstrap over 16 points has a wide interval. If the candidate follows the law, the lower bound exceeds 1 in only 11% of simulated experiments.
  - So `not_separated` is the expected primary outcome **even if the fix works as modelled**.
  - These 16 points are also the panel on which the anomaly was found. Their gain is expected to overstate the gain on new targets.
  - The 384-point held-out panel has about **94%** power at the model's 1.33×, and it has no selection bias. It is pre-registered as a secondary because the primary rule was given; it never changes the verdict.
- **Runtime estimate.** About 1.6–2.6 h, including a 15–25 min build.

Local validation (2026-10-15, cloud-agent VM; no timing claims)

The VM is a shared 4-vCPU guest, so its clocks are not evidence. The smoke receipt `smoke/` holds correctness only.

- **Build and tests.** rustc 1.97.0 here; the runner pins 1.99.0. The online crate's 6 producer tests pass, including the three new Feistel bijection tests, as do the tuned rho's 4 tests. B's factor-log table re-verifies in the pinned oracle (PASS).
- **Exactness on Q0** (direct arm runs):
  - `qB` reproduced B's 8,845,441 probes and relation [87298, 27061, 36601, 66281].
  - `qH` gave exactly the predicted 4,422,869 probes and the same relation.
  - `qP` (Feistel) found a different relation, [77273, 22971, 54962, 62539], after 1,988,184 probes.
  - All three recovered 3141592653589793, the validation sidecar.
- **Smoke run** (`smoke/`). `run_schedule.py--segmentsQ1,F1,H1` ran 17 rows through the real wrapper, gates and verifier: Q0, w00 and w01 with all five arms, and held-out slice 0..63 in both orders. All 17 rows were valid and every correctness check passed:
  - the left-major arms reproduced B's archived probes and relations;
  - the planted arms hit their predicted counts exactly;
  - all scalars agree across both orders and the tuned rho, and replay in the oracle, as do the 64 held-out seeds.
  - A dry run of the full analysis and figure scripts on that partial receipt worked; its outputs were discarded.

Result

Pending run 38786433493. The archive steps are:

1. Download artifact `n83-scan-order-<run_id>-<attempt>` into `run_github_<run_id>/`.
2. Run `python3verify_receipt.pyrun_github_<run_id>/panel`, which must reproduce `result.json`.
3. Run `python3make_figures.pyrun_github_<run_id>/panel/result.json`.
4. Fill this section:
   - the primary with its interval and verdict;
   - the held-out and Q0 ratios;
   - probes over the law and CV per arm;
   - the new baseline `S_rho` and its delta;
   - the tuned-rho and BL gaps.

Files

File	Role
protocol.json, make_protocol.py, schedule.json, make_schedule.py	frozen protocol, its builder, frozen run order
make_online_source.py, online/	scan-order producer derived from B's producer by counted exact replacements, plus A's tuned rho; reconcile's Cargo.lock
restore.py, check_log_table.py	restore and digest-check crypto 770f6a69 and every local source; re-verify the factor-log table
receipt_blocks.py, analysis/receipt_blocks.json	exact reconstruction of the 17 archived baseline scans; planted starts; the representation model
power_check.py, analysis/power.json	power of the primary and the held-out panel (model)
make_heldout_targets.py, inputs/heldout_targets.json	the 384 held-out public points
inputs/bl_run_38689817618_ic/, inputs/planted_starts.json	archived baseline receipts (copied from claude/n83-bl-arm @ 4b8ff92a, digests in protocol.json); planted-control starts
arm.py, run_schedule.py	one arm per pinned process; the schedule runner (liftscan's, adapted)
verify_receipt.py	gates, statistics, decision rule; result.json, result_table.md
make_figures.py, figures/	figure sources (TikZ), PDF and PNG renders
README.pdf	this report: pandocREADME.md-oREADME.pdf--pdf-engine=pdflatex-Hreport_header.tex--lua-filterreport_filter.lua-Vgeometry:margin=2cm-Vfontsize=10pt-Vcolorlinks=true

Limits

- All verdicts are Tier 1 and instance-scoped: one runner, one pinned CPU, not host-isolated.
- **The order changes which relation is found, so probes differ by design.** Each fresh point is one draw per arm. The primary's interval carries that target-level randomness, which is why its power is low.
- The held-out panel runs 64 targets per process. Each target has its own one-target online interval, with no shared target-dependent work, but the process is warm. The panel is secondary.
- The representation model is a heuristic estimate. It is not the reason for any verdict.
- The tuned-rho and BL gaps are distance remaining. Nothing here is a claim to beat rho.
- **Next improvements to try** after this run:
  - combine the Feistel order with lift_scan, once both are measured;
  - fold the redundant ordered states (A, B) and (B, A), which test the same pair-sum orbit. That halves the index setup, but the per-probe hit rate is unchanged, so it is a setup and memory improvement, not an online one.